

GEMM·CUDA

这一题我感觉其实就是 CPU 版本的改编，思想几乎完全一样，只是需要用 CUDA 来写

首先想法还是和 CPU 是一致的，就是每个 thread 计算 C 中的一个元素，因为 C 中一个元素需要 A 的一行和 B 的一列来计算，所以我们分配给每个 CUDA thread 的任务就是用 A 的一行每个元素乘以 B 的一列的每个元素，然后累加，乘以 alpha，这个是前半部分；而后半部分只需要把原数字乘以 beta 即可。

所以思路是非常简单的。只是需要知道点 CUDA 的语法。我们知道 CUDA 的一个线程负责一个数字，我们需要知道现在这个线程负责的数字的位置。所以计算当前数字的 row 和 col。row 的计算只需要 $\text{blockDim.y} * \text{blockIdx.y} + \text{threadIdx.y}$ 即可，col 的计算只需要 $\text{blockDim.x} * \text{blockIdx.x} + \text{threadIdx.x}$ 即可。

然后我们判断一下边界，也就是，超出(M, N)范围的我们不用算，直接 return 即可

接下来很简单了，跟 CPU 的差不多，就是计算一行乘以一列。这道题还需要用到两个宏，就是 half 和 float 转换的宏，一个是 `__half2float` 一个是 `__float2half`。

然后在调用 kernel 的时候还需要传入 grid 和 block，我让 block 取一个(16, 16)，然后 grid 就根据 block 来计算就行，向上取整。

所以最终的代码就是

```
#include <cuda_fp16.h>
#include <cuda_runtime.h>

__global__ void kernel(const half *A, const half *B, half *C, int M, int N, int K,
float alpha, float beta) {

    // 根据block和thread找到现在要计算的row和col
    int row = blockDim.y * blockIdx.y + threadIdx.y;
    int col = blockDim.x * blockIdx.x + threadIdx.x;

    // 然后还需要判断一下row, col在不在范围内
    if(row >= M || col >= N) {
        return;
    }

    // 也就是我们要算的是C[row][col]
    float total = 0;

    for(int x = 0; x < K; x++) {
        total += __half2float(A[row * K + x]) * __half2float(B[x * N + col]);
    }

    C[row * N + col] *= beta;
    // 这里得转化一下half和float
    C[row * N + col] += __float2half(alpha * total);
}

extern "C" void solve(const half* A, const half* B, half* C, int M, int N, int K,
float alpha,
float beta) {
    // 把这里改成你的实现
```

```

    dim3 block(16, 16);
    // 向上取整
    dim3 grid((N + block.x - 1) / block.x, (M + block.y - 1) / block.y);
    kernel<<< grid, block>>>(A, B, C, M, N, K, alpha, beta);
}

```

测试一下

```

nsccl@login:~$ nsccl-run bash run_tests.sh
execution d055742d-06e1-4bab-a0cb-8acb20ed3340
  ok   example_2x2x3: m=2 n=2 k=3 time_ms=0.096
  ok   unaligned_37x41x29: m=37 n=41 k=29 time_ms=0.013
  ok   square_16_beta0: m=16 n=16 k=16 time_ms=0.008
  ok   square_16_beta1: m=16 n=16 k=16 time_ms=0.007
  ok   tall_32x16: m=32 n=16 k=16 time_ms=0.008
  ok   wide_16x32: m=16 n=32 k=16 time_ms=0.007
  ok   alpha0_beta1: m=16 n=16 k=32 time_ms=0.009
  ok   perf_1024: m=1024 n=1024 k=1024 time_ms=1.217
summary cases=8 failures=0 worst_case_ms=1.217
cuda-gemm-fp16_RESULT: PASS
WARNING: Overriding HOME environment variable with APPTAINERENV_HOME is not permitted
execution d055742d-06e1-4bab-a0cb-8acb20ed3340: completed

```

没有问题，当然这题也可以像 CPU 那题进行一点优化，也就是利用 Locality 来提高 Cache Hit Rate，不过上次感觉提升不大，所以也就没有再尝试了。